

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

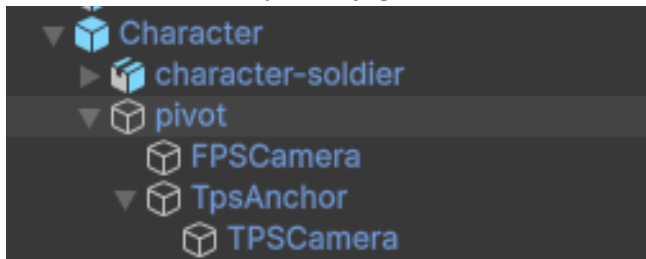
(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	Computación Gráfica, Visión Computacional y Multimedia				
TÍTULO DE LA PRÁCTICA:	<i>Controladores de entrada de primera y tercera persona</i>				
NÚMERO DE PRÁCTICA:	<i>04</i>	AÑO LECTIVO:	<i>2026</i>	NRO. SEMESTRE:	<i>2026A</i>
FECHA DE PRESENTACIÓN	<i>10/05/2026</i>	HORA DE PRESENTACIÓN	<i>23:59</i>		
INTEGRANTE (s): Chahua Charca, Luis Ayben				NOTA:	
DOCENTE(s): <i>Rene Alonso Nieto Valencia</i>					

SOLUCIÓN Y RESULTADOS																	
I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS 1. Inputs del Jugador Para los inputs del jugador (moverse, saltar, correr) se utilizó un componente character controller, para simplificar y agilizar su comportamiento ☒ Character Controller ? ⚙️ ⋮ <table style="width: 100%; border-collapse: collapse;"> <tr><td>Slope Limit</td><td style="border: 1px solid #34495e; padding: 2px 10px;">45</td></tr> <tr><td>Step Offset</td><td style="border: 1px solid #34495e; padding: 2px 10px;">0.3</td></tr> <tr><td>Skin Width</td><td style="border: 1px solid #34495e; padding: 2px 10px;">0.08</td></tr> <tr><td>Min Move Distance</td><td style="border: 1px solid #34495e; padding: 2px 10px;">0.001</td></tr> <tr><td>Center</td><td style="display: flex; justify-content: space-between;"> X 0 Y 0 Z 0 </td></tr> <tr><td>Radius</td><td style="border: 1px solid #34495e; padding: 2px 10px;">0.5</td></tr> <tr><td>Height</td><td style="border: 1px solid #34495e; padding: 2px 10px;">1.8</td></tr> <tr><td colspan="2" style="padding-top: 10px;">▶ Layer Overrides</td></tr> </table> Esto deja libre algunos métodos que nos servirán para explicar los demás scripts		Slope Limit	45	Step Offset	0.3	Skin Width	0.08	Min Move Distance	0.001	Center	X 0 Y 0 Z 0 	Radius	0.5	Height	1.8	▶ Layer Overrides	
Slope Limit	45																
Step Offset	0.3																
Skin Width	0.08																
Min Move Distance	0.001																
Center	X 0 Y 0 Z 0 																
Radius	0.5																
Height	1.8																
▶ Layer Overrides																	

2. Jerarquía de Objetos

una vez diseñado el input del jugador se establece la siguiente jerarquía de GameObjects



Se establece que a la par del modelo (“**character-soldier**”) se pone un pivote el cual es el punto sobre el cual girarán las 2 cámaras del juego.

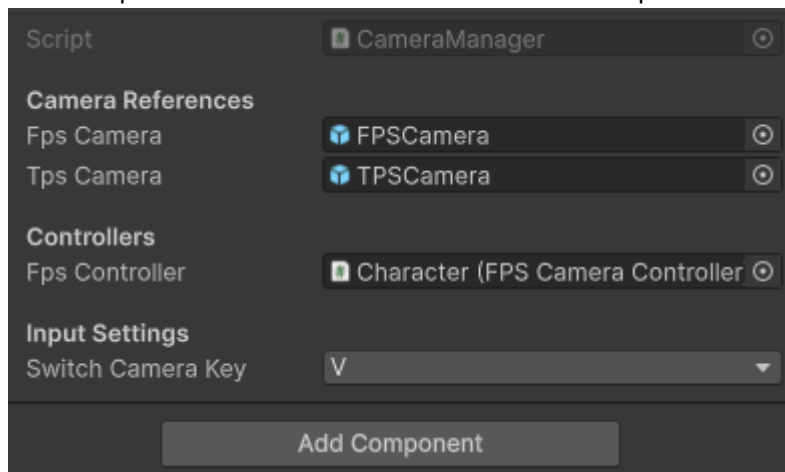
Mientras tanto la cámara principal es FPSCamera (Nombrada así por First Person Shooter Camera) es la que inicia el juego.

También está el GameObject TpsAnchor: el cual es el contenedor de la TPSCamera (Nombrada así por Third Person Shooter Camera), que a su vez contiene el Script de “**CameraCollision.cs**”

En cuanto al objeto “**Character**”, tiene asignados a si 2 scripts importantes “**CameraManager.cs**” y “**FPSCameraController**”

3. CameraManager.cs

El script que permite la activación de los 2 tipos de cámara que existen , ya que recibe los GameObject de los 2 tipos de cámara así como la tecla de entrada para cambio de cámara, en este caso usamo “**V**”.



```

CSharp
void HandleCameraSwitch()
{
    if (Input.GetKeyDown(switchCameraKey))
    {
        isFirstPerson = !isFirstPerson;

        if (isFirstPerson)
        {

```

```
        fpsCamera.tag = "MainCamera";
        tpsCamera.tag = "Untagged";
        ActivateFPS();
    }
    else
    {
        tpsCamera.tag = "MainCamera";
        fpsCamera.tag = "Untagged";
        ActivateTPS();
    }
}

//funcion que activa FPS
void ActivateFPS()
{
    fpsCamera.SetActive(true);
    tpsCamera.SetActive(false);

    fpsController.enabled = true;
}

//funcion que activa TPS
void ActivateTPS()
{
    fpsCamera.SetActive(false);
    tpsCamera.SetActive(true);

    fpsController.enabled = true;
}
```

4. FPSCameraController.cs

Camera Controller, es el script encargado del comportamiento del jugador relacionado con la cámara, al estar gestionado por el Character controller, nos resulta más fácil delegar las tareas de control de cámara a este script, cuyo funcionamiento es el siguiente

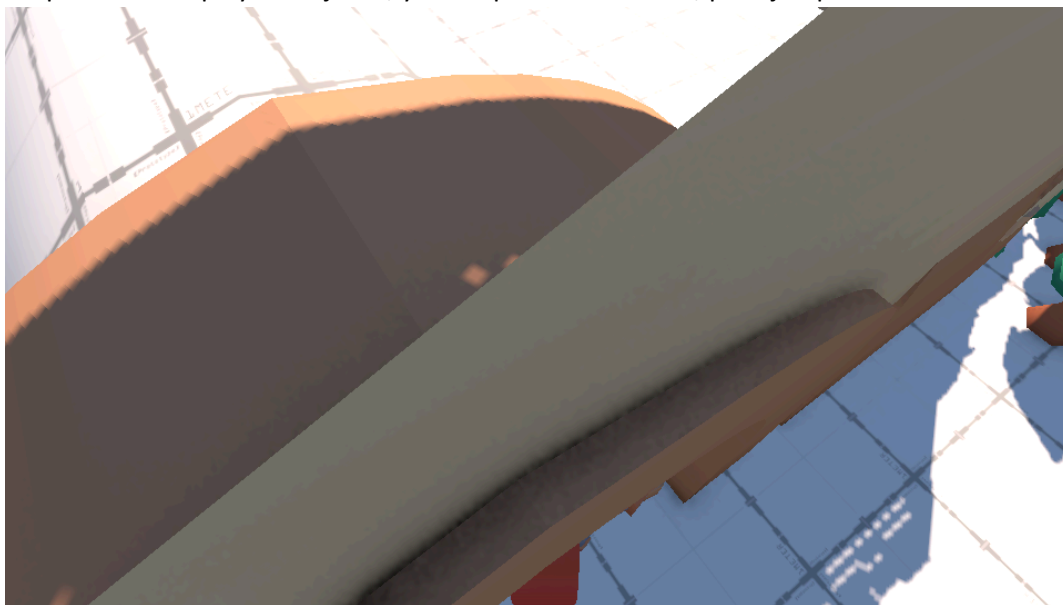
- Recibe el Objeto Jugador y el Pivote de cámara así como valores para la sensibilidad del mouse y el rango de rotación máximo y mínimo
- Ejecuta **HandleMouseLook()** para tomar la entrada del mouse del jugador y así girar en tiempo real usando un Quaternion

CSharp

```
void HandleMouseLook()  
{  
    float mouseX = Input.GetAxis("Mouse X") * mouseSensitivity;  
    float mouseY = Input.GetAxis("Mouse Y") * mouseSensitivity;  
  
    // Rotación horizontal del jugador  
    playerBody.Rotate(Vector3.up * mouseX);  
  
    // Rotación vertical del pivot  
    verticalRotation -= mouseY;  
  
    verticalRotation = Mathf.Clamp(  
        verticalRotation,  
        -upDownRange,  
        upDownRange  
    );  
  
    cameraPivot.localRotation =  
        Quaternion.Euler(verticalRotation, 0f, 0f);  
}
```

5. CameraCollision.cs

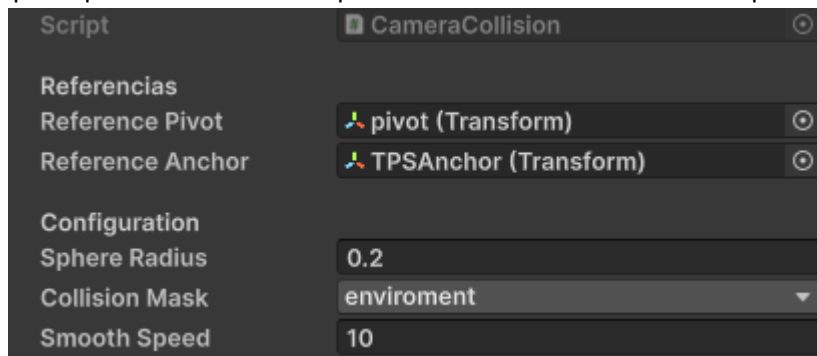
Dentro de la implementación de estas cámaras surgen problemas muy concretos, tal es el caso de las colisiones con la visión de la cámara, por ejemplo cuando algo está detrás de una pared, la cámara en 3ra persona lo tapa y no deja ver, y así se pierde inmersión, por ejemplo:



Jugador detrás de la pared, a punto de desaparecer, además la cámara traspasa la pared.

Lo cual no debería pasar, para solucionar este problema, se implementó el Script **CameraCollision.cs** que trabaja de la siguiente manera:

- Recibe los elementos pivote, TPSAnchor (contenedor de la cámara 3ra persona) y un LayerMask que le permitiera identificar que elementos no deberían ser traspasables.



- Posterior a eso, Ejecuta LateUpdate() que permite lanzar un raycast desde el pivote hasta el anchor, lo que permite calcular si habrá o no habrá colisiones entre estos 2 para transformar la posición de la cámara, a una velocidad constante para mantener la suavidad, y mantener su rotación.

CSharp

```
void LateUpdate()
{
    Vector3 targetPosition = referenceAnchor.position;

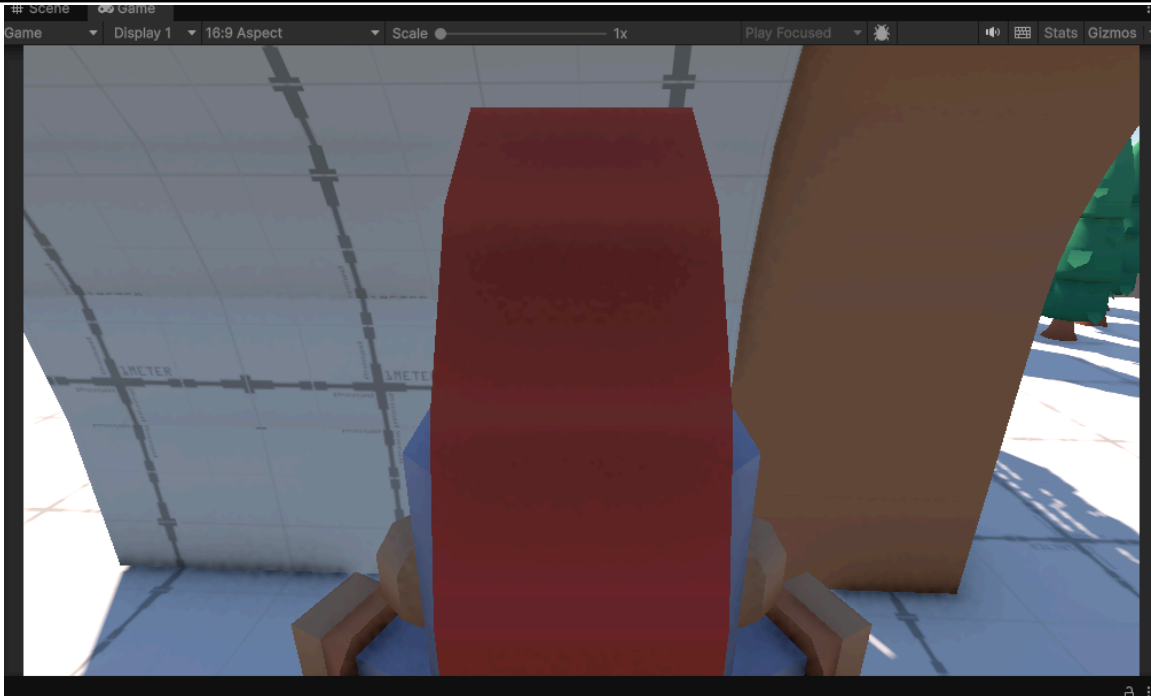
    Vector3 direction = (targetPosition -
referencePivot.position).normalized;
    float maxDistance = Vector3.Distance(referencePivot.position,
targetPosition);

    RaycastHit hit;

    if (Physics.SphereCast(referencePivot.position, sphereRadius, direction,
out hit, maxDistance, collisionMask))
    {
        targetPosition = referencePivot.position + direction * hit.distance;
    }

    transform.position = Vector3.Lerp(transform.position, targetPosition,
Time.deltaTime * smoothSpeed);

    transform.rotation = referenceAnchor.rotation;
}
```



en lugar de pasar por detrás de la pared, se ajusta para poder seguir viendo al jugador.

II. SOLUCIÓN DEL CUESTIONARIO

1. *¿Qué problemas pueden presentar las cámaras de tercera persona?*
El problema de las colisiones en el entorno que pueden generar problemas con seguir al jugador, o permitir su paso a través de las paredes.
2. *¿Cómo han realizado el control de múltiples cámaras en la escena?*
Hice un Script llamado Camera Manager que permite hacer el Switch entre cámaras al presionar V

III. CONCLUSIONES

En conclusión, el manejo de distintos tipos de cámaras es vital para desarrollar experiencias más inmersivas además de gratificantes, fue una experiencia bastante demandante además de grata

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA



UNIVERSIDAD NACIONAL DE SAN AGUSTIN
FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA



Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación

Aprobación: 2022/03/01

Código: GUIA-PRLE-001

Página: 7

--